

CityPulse: Intelligent Parking & Traffic Control Platform

Review of CSE499A and Plan for CSE499B

CSE499B — Section 15 — Project Group 03

Md. Rahad Hossain
ID: 2211316642

Md. Yousuf
ID: 2211461642

MD. Rokib Hasan Oli
ID: 2211950642

Safayat Ibrahim
ID: 2131174642

North South University, Dhaka, Bangladesh

Abstract—CityPulse is a parking and traffic platform for Dhaka that connects drivers, parking owners, and traffic authorities. In CSE499A we trained a MobileNetV2 occupancy classifier that reaches 99.18% accuracy on a parking lot it never saw during training, and built a working platform around it: role-based authentication, space registration, booking, a driver app, and an owner portal.

The model has not yet been tested on Bangladeshi footage, camera streams are not processed live, and several authority, web, and mobile features still rely on demonstration data or lack backend support.

This report reviews CSE499A and sets out the CSE499B plan, with four technical objectives: a Dhaka parking dataset with a third fine-tuning phase, live stream processing, Bangla licence plate recognition feeding a live authority dashboard, and demand-aware pricing driven by live occupancy. The web, mobile, and backend features left unfinished in CSE499A will also be completed.

Index Terms—smart parking, parking occupancy detection, transfer learning, Bangla licence plate recognition, dynamic pricing, urban mobility, Bangladesh

I. INTRODUCTION

Dhaka’s average traffic speed has fallen to around 7 km/h [1], and unorganized roadside parking is one of the main causes. Deploying traffic police at every busy point cannot scale across the city. With over 180 million mobile subscribers in Bangladesh [2], a mobile-first digital solution can reach most drivers.

CityPulse was proposed in CSE499A as a platform that uses existing CCTV cameras to detect free parking slots automatically and deliver that information to drivers, owners, and authorities. CSE499A delivered the core detection model and the platform around it. CSE499B completes the system: it validates the model under Dhaka conditions, makes the pipeline live, and adds the enforcement and pricing features that owners and authorities need.

II. PROBLEM STATEMENT

The key problems that CityPulse addresses are:

- **Illegal roadside parking:** organized parking is invisible to drivers, so they stop on the road.
- **No real-time information:** no deployed system shows free parking before arrival.
- **Manual updates do not scale:** local services rely on operators typing availability.

- **No automated vehicle identification:** authorities must be present to enforce.
- **Fixed pricing:** busy lots fill up while nearby ones stay empty.

III. RELATED WORK

Parking occupancy detection from fixed cameras is a mature field. PKLot [3] and CNRPark+EXT [4] are the standard benchmarks.

VPS-Net [5] detects and classifies slots in bird’s-eye images from car-mounted cameras. The most relevant local work is that of Prova et al. [6] at NSU. Their classifier, trained only on international data, reaches 86.1% (indoor) and 90.5% (outdoor) on Dhaka CCTV footage. They name fine-tuning on local data and manual slot marking as open problems. Existing Bangladeshi services, DNCC Smart Parking in Gulshan [7] and Parking Koi [8], rely on availability updated by hand and do not use cameras.

For Bangla licence plates, Ashrafee et al. [9] released the first public datasets, 1,000 plate images in BRTA format and 79 Dhaka traffic videos covering 98 plates. Their CPU pipeline uses a Haar-cascade filter to skip empty frames, a MobileNet SSDv2 plate detector, and a cloud OCR API, keeping the three most confident frames per vehicle; it reaches 82.7% video detection and 60.8% OCR F1 at 27.2 FPS. Hasin et al. [10] train YOLOv8m in two stages, spatial and then photometric augmentation with progressive layer unfreezing, on 6,517 annotated images. Their OCR pairs a ViT encoder with a Transformer decoder and the BanglaBERT tokenizer, trained on 13,629 plates from all 64 districts (102 character classes). They reach 97.83% localization and a 13.23% character error rate (CER), validated on 276 unseen toll-booth CCTV images at night and in bad weather.

IV. REVIEW OF CSE499A

A. Proposal Objectives versus Outcome

Table I checks each objective of the CSE499A proposal against what was delivered. Two items proposed only as future work were also built: slot booking with reservation-aware availability, and a reproduction of VPS-Net.

B. Occupancy Model

PKLot and CNRPark+EXT were merged into 808,991 labelled slot images from 12 cameras (published on Kaggle).

TABLE I
CSE499A PROPOSAL OBJECTIVES AND OUTCOME

Objective (499A proposal)	Status	Evidence / gap
1. Driver app: search, find, navigate	Done	Map search, booking, Google Maps hand-off
2. CV module on live CCTV feeds	Partial	99.18% model; still images only, not live streams
3. Owner registration portal	Done	Space, photo, camera onboarding on live API
4. Real-time updates to the app	Partial	10-min polling; no push
5. Authority occupancy view	Partial	UI built; no backend data
6. Push notifications	Not done	Schema only
7. LPR / jam study, datasets	Partial	Literature surveyed; no plates collected
8. Architecture design	Done	Architecture document, 21-model schema, OpenAPI

The UFPR05 lot was held out entirely for validation, so accuracy reflects a facility the model has never seen.

MobileNetV2 [11] with a new two-class head was trained in two phases on a Tesla T4, as shown in Table II. Phase 2 reduced the error rate from 2.72% to 0.82%.

TABLE II
TWO-PHASE TRAINING (VALIDATION ON UNSEEN LOT UFPR05)

Phase	Trainable	LR	Best val. acc.
1: head only	head	10^{-3}	97.28% (ep. 5)
2: full fine-tune	all	10^{-5}	99.18% (ep. 9)

VPS-Net’s slot-occupancy classifier was reproduced on ps2.0 at 98.53% (9.01 MB, 0.60 ms per image). Its slot-detection stage was not reproduced.

C. Backend and AI Service

The NestJS backend manages identity, spaces, bookings, and scheduling, with JWT and permission-based access for the driver, owner, and authority roles. The FastAPI service serves the models and logs each prediction to MongoDB. A scheduled job sends slot images to the model and marks a slot occupied only when confidence is at least 0.90.

Availability combines the model output with bookings: $\text{unavailable} = \text{booked}_{\text{now}} \vee \text{occupied}_{\text{visual}}$. A reserved slot is therefore never offered twice.

Prices are a flat base rate $\times$ hours; the pricing and payment tables have no service yet.

D. Mobile App

The driver app is built in Flutter with a layered structure: screens and widgets, Provider for state, a service layer (API, authentication, session, booking, location, places, profile), and data models. Drivers stay signed in between launches, see nearby parking on a map, view a space’s details and price in BDT, book a slot for a chosen duration, and list or cancel

bookings, all against the live backend. Navigation hands off to Google Maps.

E. Web Portal

The owner and authority portals share one Next.js application (React, TypeScript, Tailwind CSS) with role-based routing and token refresh. Sign-up, sign-in, and parking-lot registration are connected to the backend. The owner camera, revenue, and settings pages and the authority command centre (zones, enforcement, analytics) are fully designed but still show demonstration data. Fifteen unit tests cover the API client, session, and routing modules, and the portal is deployed publicly.

F. Limitations Carried Forward

CSE499A left six limitations, which define CSE499B: (1) no Bangladeshi evaluation data; (2) still images instead of video streams; (3) licence plate recognition surveyed only; (4) several web and mobile features use demonstration data or have no backend service; (5) the booking lifecycle, account recovery, and backend hardening are incomplete; and (6) no precision, recall, latency, or load testing.

V. OBJECTIVES FOR CSE499B

- 1) Collect and label a Dhaka parking dataset. Measure how well the current model transfers to it, then fine-tune a third phase on Dhaka data.
- 2) Process RTSP camera streams continuously and push slot changes to clients in real time.
- 3) Detect and read Bangla licence plates from the same camera streams.
- 4) Connect the authority dashboard to live occupancy and plate data, with privacy controls.
- 5) Implement demand-aware pricing driven by live occupancy.
- 6) Complete every item left pending in CSE499A (Table III), including push notifications, payments, social sign-in, and in-app navigation, and connect all web and mobile features to backend services.
- 7) Evaluate the full system: precision, recall, character error rate, latency, and load.

Table III lists every CSE499A item that was not finished, from the proposal objectives, the sprint plan, and the database schema, and where it is completed in CSE499B.

VI. SYSTEM OVERVIEW

The service-based architecture from CSE499A is kept. CSE499B adds two components in the AI service, a stream worker and a plate recognition module, plus a pricing module in the backend (Fig. 1). Slot events travel over Redis Pub/Sub and reach clients through WebSocket, as specified in the CSE499A architecture document. New technology is limited to OpenCV for stream ingestion and YOLOv8 with a Bangla OCR model for plates; the CSE499A stack is otherwise unchanged.

TABLE III
PENDING CSE499A ITEMS COMPLETED IN CSE499B

Pending from CSE499A	CSE499B work
CV on live CCTV feeds	RTSP stream worker (VII-B)
Real-time updates to clients	Redis and WebSocket push to web and mobile
Authority occupancy view	Live command centre: occupancy, zones, enforcement, analytics
Push notifications	Notification service; booking and availability alerts on mobile
Navigation to parking entrance	In-app route guidance (route preview exists)
Dhaka parking images	Dhaka dataset and Phase 3 fine-tuning (VII-A)
Bangla LPR study and plate images	Plate detection and OCR on live streams (VII-C)
Pricing and payment schema	Dynamic pricing (VII-E); bKash/Nagad sandbox payments
Social sign-in schema	Google sign-in on web and mobile
Owner portal live data, file storage	Live owner views; object storage for uploads
End-to-end integration and testing	Automated tests and full evaluation (Sprint 6)

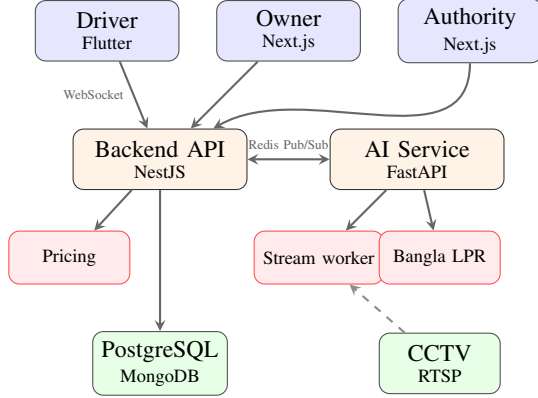


Fig. 1. CityPulse architecture. Components added in CSE499B are shown in red.

VII. FEATURES BREAKDOWN

A. Dhaka Dataset and Phase 3 Fine-Tuning

We will collect footage from two to three Dhaka parking facilities in day, night, and rain, and label 3,000–5,000 slot images. The Phase 2 model is first tested on this data *without* retraining, to measure how well it transfers compared with the 86–90% baseline of [6]. A third training phase then fine-tunes on Dhaka data, with one site held out for validation. We will report accuracy, precision, recall, F1, and the confusion matrix, and recalibrate the 0.90 confidence threshold on this data.

B. Live Slot Availability

The stream worker samples frames from each camera, crops each slot, and sends the crops to the model. It keeps the camera status (active or offline) up to date. Changed slots are published over Redis and pushed to the driver app over WebSocket, replacing the 10-minute polling.

C. Bangla Licence Plate Recognition

Training data combines the public image and video sets of [9], the datasets of [10] where available, and 100–200 plate images we collect in Dhaka. The YOLOv8 detector follows the two-stage schedule of [10]. The OCR step reads the two-line Bangla format (district and vehicle class on top, serial number below) with a ViT and BanglaBERT model, using TrOCR as a baseline. On live streams we keep the three most confident frames per vehicle, as in [9]. We will report detection mAP, character error rate, and full-plate accuracy. Each plate reading is stored in MongoDB with the time, camera, and confidence score.

D. Authority Dashboard and Privacy

The authority views get live data for occupancy, zones, and a searchable enforcement log. Following the legal review in CSE499A, plate data will be visible only to the authority role, logged on every lookup, and deleted after a fixed retention period. As a stretch goal, a plate detected in a booked slot with no matching booking will be flagged as unauthorised use.

E. Dynamic Pricing

The pricing module uses the existing `pricing_rules` table. Owners set a base rate per slot type (standard, motorcycle, disabled, EV) and separate peak and off-peak rates. When live occupancy o reaches the threshold τ (default 80%), the hourly rate becomes

$$r = \min(r_{\max}, \max(r_{\min}, r_{\text{base}} \cdot f)), \quad (1)$$

with surge factor $f = 1.5$ by default. Below the threshold, the base rate applies. The price is fixed at the moment of booking, and the owner’s revenue page is connected to booking data.

F. Backend Completion

The backend gains the services that the clients are waiting on: account recovery, Google sign-in, camera management, notifications, sandbox payments, and endpoints for authority and revenue data. The booking lifecycle will be completed, including conflict checks between overlapping reservations and closing bookings once they end. Uploaded files move to object storage, and the API is hardened with stricter access rules, rate limiting, and automated tests.

G. Mobile App

The driver app gains live availability over WebSocket and surge prices fixed at booking time, plus push notifications, in-app route guidance, Google sign-in, sandbox payment, password recovery, profile editing, and server-side booking history. Configuration and API keys move out of the source code, and widget and integration tests are added.

H. Web Portal

The owner and authority views are connected to live back-end data in place of their demonstration data: occupancy, zones, and the enforcement log with plate search for authorities; cameras, revenue, and pricing rules for owners. Profile management, password recovery, and Google sign-in are completed, and end-to-end tests are added.

VIII. COMPLEX ENGINEERING PROBLEM AND ACTIVITIES

Table IV maps the project to the BAETE attributes: P1, five of P2–P7, and three of A1–A5.

TABLE IV
COMPLEX ENGINEERING PROBLEM (P) AND ACTIVITY (A) MAPPING

ID	How CityPulse addresses it
P1	Depth of knowledge (K3–K6, K8): databases and networks, transfer learning and detection, service design, modern tools, and extension of the research literature
P2	Conflicting requirements: accuracy vs. inference cost; live vision vs. booking state; surveillance vs. privacy; owner revenue vs. driver cost
P3	Depth of analysis: classification vs. detection, cross-lot evaluation, phased fine-tuning; transfer from international benchmarks to Dhaka footage
P4	Infrequent issues: two-line Bangla plates, unmarked Dhaka lots, no public local data
P6	Stakeholders: drivers, owners, and authorities with conflicting data-access needs
P7	Interdependence: stream, occupancy, LPR, backend, pricing, and the web and mobile clients all depend on each other
A1	Resources: team, 0.8M-image dataset, GPUs, CCTV cameras, field data
A3	Innovation: extends [6] with $10\times$ data, cross-lot evaluation, and Dhaka fine-tuning; pricing driven by the vision model
A4	Consequences: less circling and roadside parking (SDG 11); privacy risk reduced by design

IX. IMPLEMENTATION PLAN

Each sprint follows the explore-then-build approach; platform-completion work runs alongside the AI/ML work.

Sprint 1 (W1–2): Review & Data Collection. *Explore:* RTSP access at Dhaka sites; public Bangla plate datasets. *Build:* review report; footage collection; stream worker reading one camera.

Sprint 2 (W3–4): Dhaka Dataset & Live Stream. *Explore:* labelling tools, frame sampling rate. *Build:* labelled Dhaka slot images and transfer test; Redis-to-WebSocket push; pricing endpoints.

Sprint 3 (W5–6): Phase 3 & Platform Completion. *Explore:* fine-tuning on a small local dataset. *Build:* Phase 3 results; booking lifecycle and account recovery; web and mobile views on live data.

Sprint 4 (W7–8): Bangla LPR. *Explore:* Bangla OCR models. *Build:* plate detector (mAP); OCR (CER); LPR running on live streams.

Sprint 5 (W9–10): Authority & Pricing. *Build:* live authority views with privacy controls; peak and surge pricing; sandbox payments; notifications; revenue view.

Sprint 6 (W11–12): Evaluation & Demo. *Build:* full evaluation (metrics, latency, load test); bug fixes; demo video; final report.

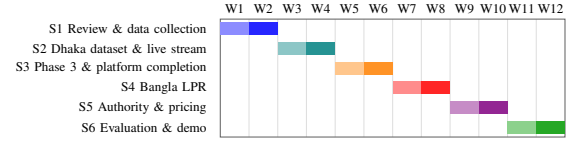


Fig. 2. CSE499B sprint plan. In each sprint the lighter half is the explore week and the darker half is the build week.

X. RISKS AND MITIGATION

If a site refuses camera access, we record our own footage. Demo-blocking features (booking lifecycle, live portal data) are built before the other CSE499A items. Weak Bangla OCR is handled by reading the two plate lines separately and fine-tuning. Real-time load is limited by sampling one frame every 2–3 s and pushing only changed slots. Plate data follows Section VII-D and the Personal Data Protection Ordinance (2025).

XI. FUTURE WORK

Beyond CSE499B: live payment settlement, availability prediction from stored occupancy history, and more facilities across Dhaka.

XII. CONCLUSION

CSE499A detected parking occupancy at 99.18% accuracy on an unseen lot and delivered a working platform for drivers and owners. CSE499B completes the system: it measures accuracy on Dhaka data, processes camera streams live, gives authorities plate recognition with privacy built in, adds pricing driven by live occupancy, and completes the web, mobile, and backend features left unfinished in CSE499A.

REFERENCES

- [1] World Bank, *Dhaka: Improving Living Conditions in the City*. Washington, DC: World Bank Group, 2018.
- [2] Bangladesh Telecommunication Regulatory Commission, *Mobile Phone Subscribers Data*, 2024. [Online]. Available: <https://www.btrc.gov.bd>
- [3] P. R. L. de Almeida, L. S. Oliveira, A. S. Britto Jr., E. J. Silva Jr., and A. L. Koerich, “PKLot – A robust dataset for parking lot classification,” *Expert Syst. Appl.*, vol. 42, no. 11, pp. 4937–4949, 2015.
- [4] G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini, and C. Vairo, “Deep learning for decentralized parking lot occupancy detection,” *Expert Syst. Appl.*, vol. 72, pp. 327–334, 2017.
- [5] W. Li, L. Cao, L. Yan, C. Li, X. Feng, and P. Zhao, “Vacant parking slot detection in the around view image based on deep learning,” *Sensors*, vol. 20, no. 7, p. 2138, 2020.
- [6] R. R. Prova, T. Shinha, A. B. Pew, and R. M. Rahman, “A real-time parking space occupancy detection using deep learning model,” in *Proc. IEEE IEMTRONICS*, 2022.
- [7] Dhaka North City Corporation, “Smart parking: DNCC debuts app in Gulshan,” *Press Xpress*, Nov. 2023.
- [8] R. Rahman, “Building a parking solution for Bangladesh,” *Future Startup*, Jul. 2019.
- [9] A. Ashrafee, A. M. Khan, M. S. Irbaz, and M. A. A. Nasim, “Real-time Bangla license plate recognition system for low resource video-based applications,” *arXiv:2108.08339*, 2021.
- [10] N. Hasin, M. A. R. Nishat, M. Islam, K. S. A. Hasan, and A. Newaz, “A robust deep learning framework for Bangla license plate recognition using YOLO and vision-language OCR,” *arXiv:2603.10267*, 2026.
- [11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510–4520.